# Generative and Adaptive Sampling Strategies for Neural Network Function Approximation in $L^2[0,1]$

May 3, 2026

**Abstract**

We compare **fifteen** sampling strategies for training a small neural network to approximate diverse 1D test functions on $[0,1]$ under a fixed sampling budget. The strategies span fixed-grid (uniform, Chebyshev, QMC), adaptive empirical (residual-based refinement), generative (adversarial, normalising flow, MDN, diffusion / score matching, importance sampling), particle-based (SVGD), uncertainty-based (deep ensemble, GP-UCB), and policy-gradient (policy / meta-sampler) approaches. For each (function, algorithm) pair we record the $L^2$ approximation error as a function of the number of sampling points, revealing convergence rates and the point-allocation patterns that drive them.

## 1 Problem setting

Let $f : [0,1] \to \mathbb{R}$ be a target function known only through pointwise evaluations. We are given a total budget $N$ of function evaluations and a parametric approximator $u_\theta : [0,1] \to \mathbb{R}$ (here, a small MLP). The goal is to choose evaluation locations $\{x_i\}_{i=1}^N \subset [0,1]$ such that the $L^2$ error

$$\mathcal{L}(\theta) = \int_0^1 \big(u_\theta(x) - f(x)\big)^2 \, dx \tag{1}$$

is minimised. The integral is estimated by trapezoidal quadrature on a dense uniform grid $\{x_k^{\text{ref}}\}_{k=1}^{N_{\text{ref}}}$, where $N_{\text{ref}}$ is chosen individually per function to resolve its finest features.

### 1.1 Approximator architecture

Unless otherwise noted, the approximator is a 4-hidden-layer MLP with $[32, 32, 32, 32]$ neurons and tanh activation ($\sim 3.2$K parameters). It is deliberately small so that sampling quality dominates the error. Alternative architectures (e.g., the MMNN with frozen random expansion layers) can be supplied via the `--model` or `--model-factory` API.

### 1.2 Test-function library and downstream tasks

The library contains 13 hand-crafted 1D functions plus a programmatically-generated library of up to $200 \times 3 = 600$ $L^2$-normalised functions partitioned into three classes:

- **Smooth:** polynomials, trigonometric polynomials, rationals, exponentials — slowly varying throughout $[0,1]$.
- **Oscillatory:** chirps, amplitude- and frequency-modulated sinusoids, multi-frequency superpositions — high-frequency content everywhere.
- **Sharp:** Gaussian bumps, mollified steps, boundary layers, cusp-like singularities — smooth except for a few narrow features.

Every function is $L^2$-normalised, i.e. $\int_0^1 f^2\,dx \approx 1$. Figure 3 shows a representative sample from the sharp class.

The sampling algorithms are decoupled from the downstream task via a `DownstreamTask` abstract interface. Three tasks are implemented:

- **Function Approximation** — fit $u_\theta \approx f$ in $L^2$; pointwise loss $|u_\theta(x) - f(x)|^2$.
- **Poisson PINN** — solve $-u'' = f$ on $[0,1]$ with $u(0) = u(1) = 0$ (hard-constrained via $x(1-x) \cdot \mathrm{NN}(x)$); pointwise loss $|-u''_\theta(x) - f(x)|^2$.
- **Deep Ritz** — minimise $E(u) = \int [\frac{1}{2}|u'|^2 - fu]\,dx$; pointwise energy density as loss.

The ground-truth solution for the PDE tasks is computed by linear finite elements on a fine mesh. All methods query $f$ at arbitrary $x$ without budget restriction; only the $N$ points used to train the approximator count toward the budget.

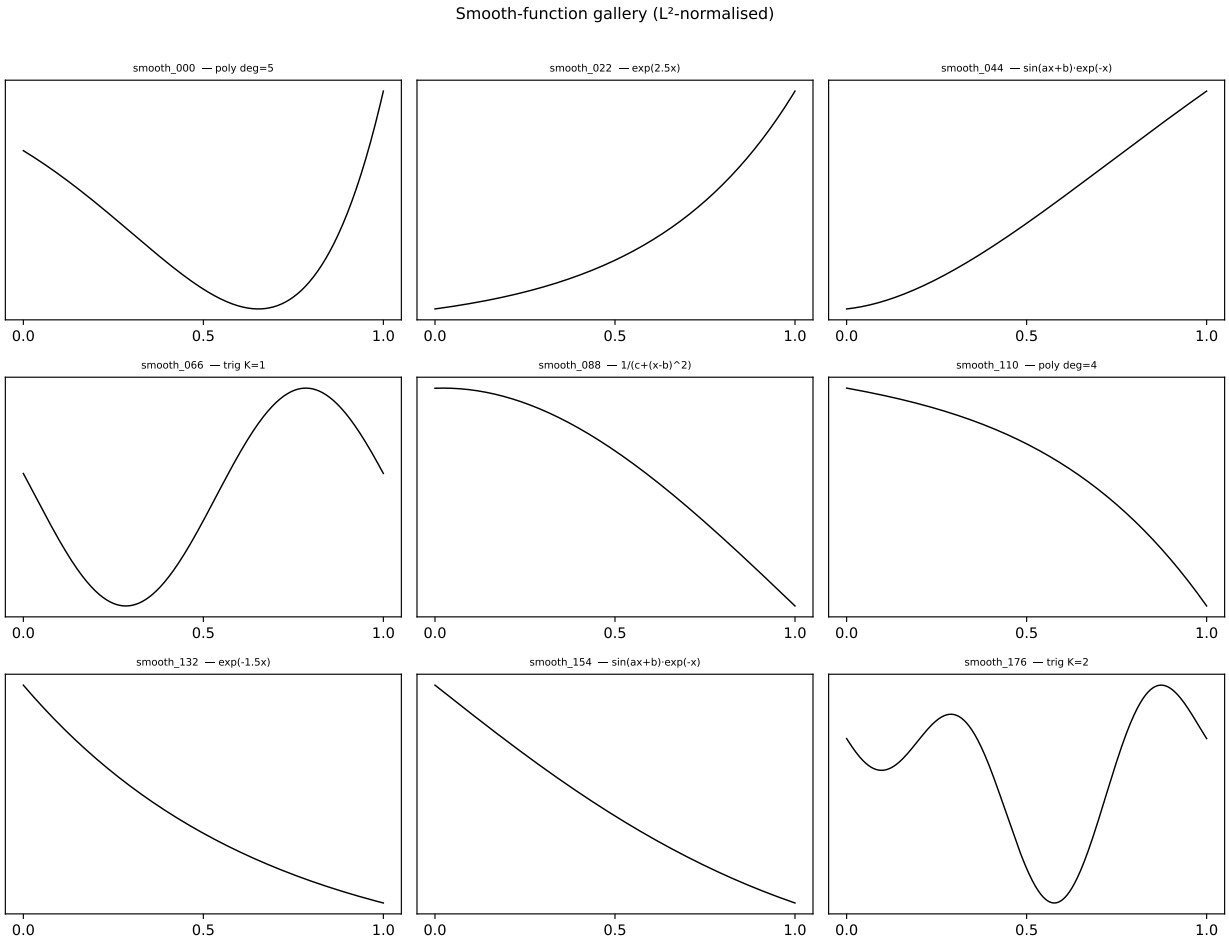


Figure 1: Representative smooth functions ($\mathbf{L^2}$-normalised).

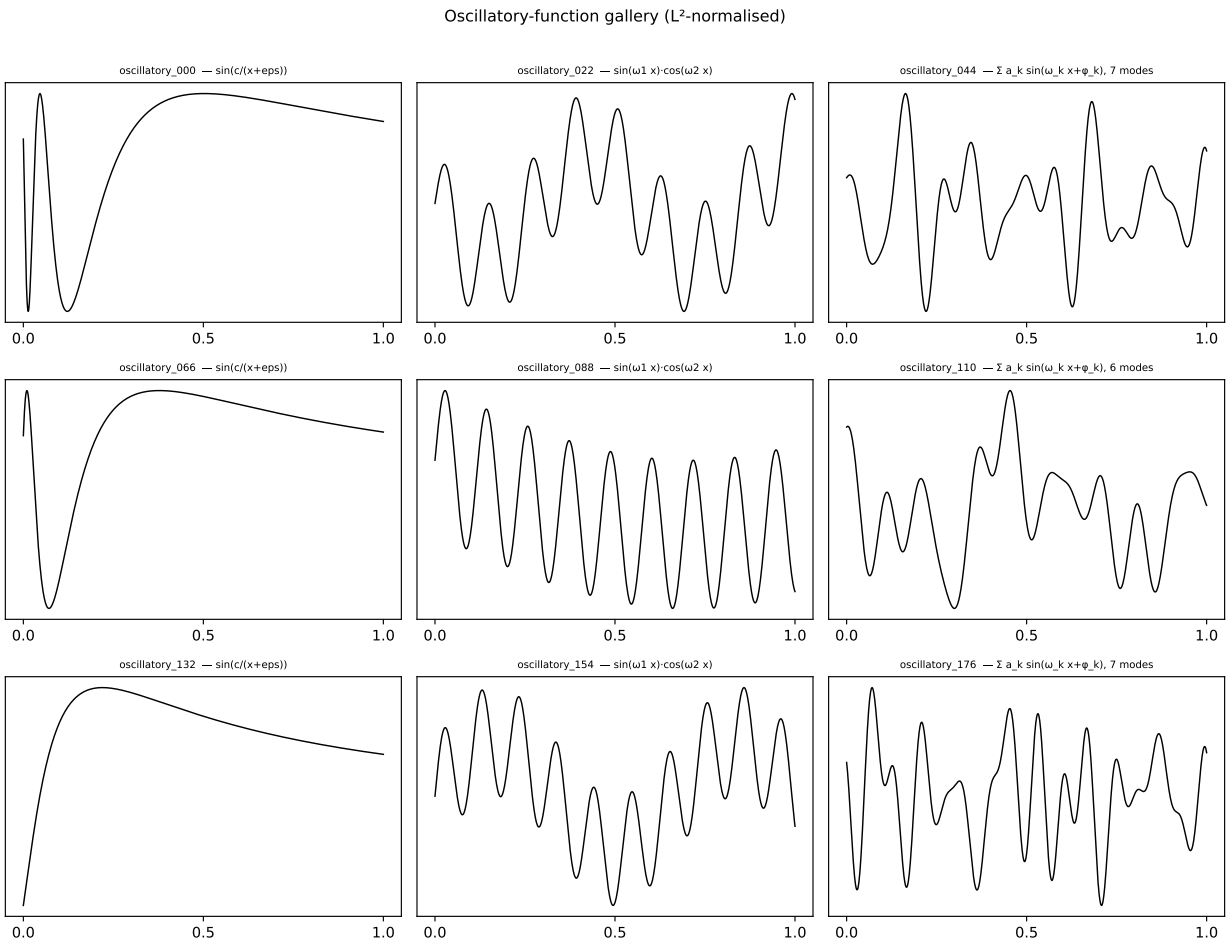


Figure 2: Representative oscillatory functions ($L^2$-normalised).

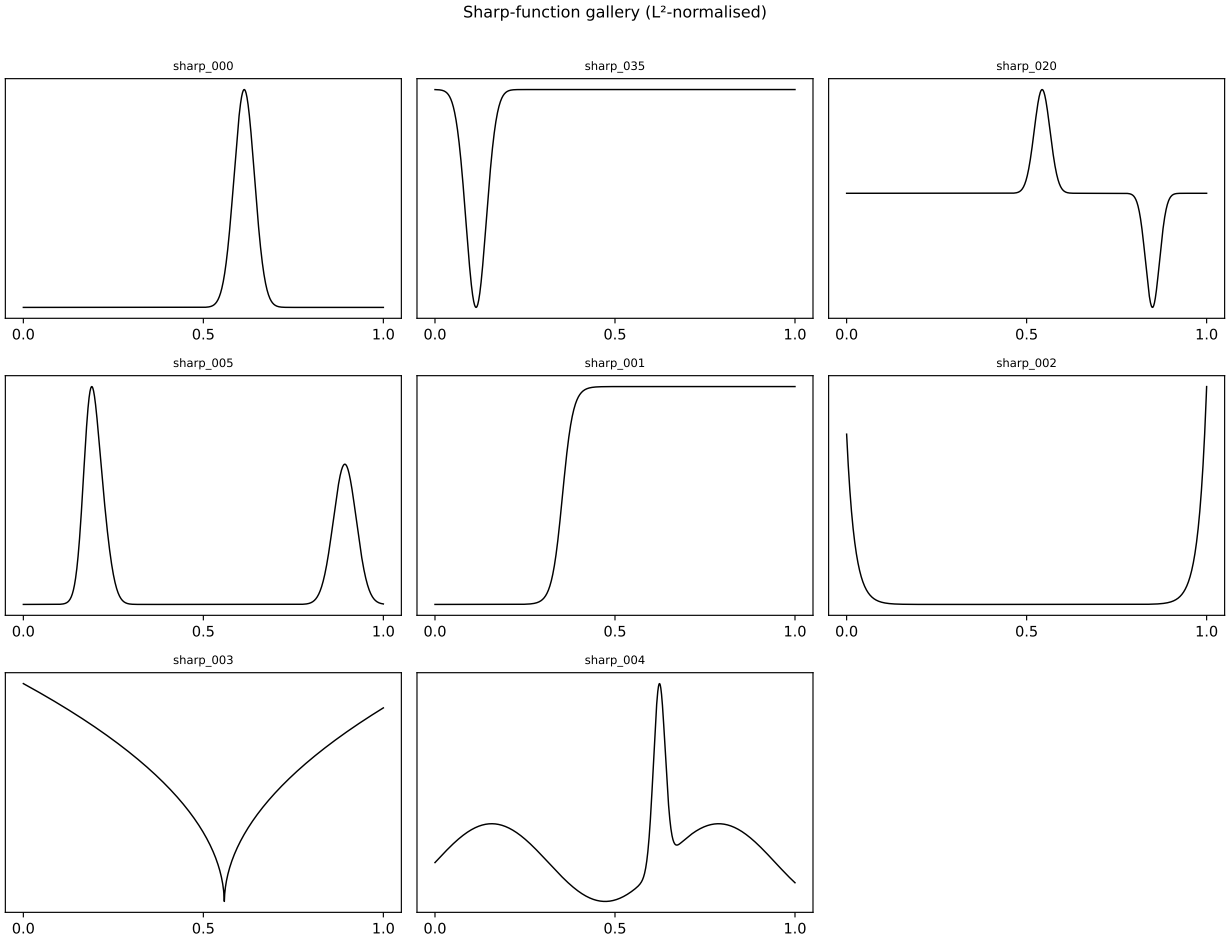


Figure 3: Representative sharp functions ($\mathrm{L}^2$-normalised).

# 2 Sampling algorithms

We organise the 14 algorithms into five families. For each we state the underlying mathematical principle and, where available, the theoretical guarantee.

## 2.1 Fixed-grid methods

**Uniform.** Equispaced grid $x_i = \frac{i-1}{N-1}$, $i = 1, \ldots, N$. The quadrature error for smooth integrands decays as $O(N^{-k})$ where $k$ depends on the smoothness of $|u_\theta - f|^2$ [4].

**Chebyshev.** Chebyshev nodes of the second kind, $x_i = \frac{1}{2}\big(1+\cos\frac{(2i-1)\pi}{2N}\big)$. For functions analytic in a neighbourhood of the Bernstein ellipse, the polynomial interpolation error decays exponentially [23]. This motivates clustering near the boundaries.

**QMC Sobol' / Halton.** Low-discrepancy sequences. The Koksma–Hlawka inequality bounds the quadrature error by $D_N^* \cdot V(f)$, where $D_N^*$ is the star discrepancy and $V(f)$ the Hardy–Krause variation. For Sobol' sequences $D_N^* = O(N^{-1}\log^d N)$ [16].

**Guarantee (Sobol').** If $g = |u_\theta - f|^2$ has bounded variation in the sense of Hardy–Krause, then $\big|\int_0^1 g(x)\,dx - \frac{1}{N}\sum_{i=1}^N g(x_i)\big| = O(N^{-1}\log N)$ vs. $O(N^{-1/2})$ for Monte Carlo [6]. Sobol' sequences require $N$ to be a power of two for optimal balance properties; our experiments respect this constraint.

## 2.2 Adaptive empirical methods

**Adaptive Residual.** Greedy refinement: begin with a coarse uniform grid, train $u_\theta$, evaluate the pointwise error on a dense candidate set, and add the $k$ points with largest error. Retrain (warm-started) and repeat until the budget is exhausted. This is the 1D analogue of the *greedy algorithm* from reduced-basis methods [2].

**Guarantee.** Under mild regularity assumptions the greedy algorithm is *quasi-optimal*: with budget $N$ it achieves the same asymptotic rate as the best $N$-point approximation from the dictionary produced by the network, up to a constant factor [5]. In practice it concentrates samples in regions of large residual.

## 2.3 Generative methods (learnable sampling distributions)

The core idea is to train a *proposal distribution* $q_\phi(x)$ so that the variance of the gradient estimator for $\theta$ is minimised. The optimal proposal $q^*(x) \propto |u_\theta(x) - f(x)|$, which is unknown, so a joint min-max or score-matching procedure approximates it.

### 2.3.1 Adversarial (REINFORCE)

A piecewise-constant generator $G_\phi$ and the approximator $u_\theta$ are trained adversarially:

$$\min_\theta \max_\phi \; \mathbb{E}_{x\sim p_\phi}\big[(u_\theta(x) - f(x))^2\big]. \tag{2}$$

The generator gradient is estimated via the score-function (REINFORCE) estimator with a moving-average baseline [24]. An entropy bonus prevents the generator from collapsing to a single bin. **The approximator $u_\theta$ is trained with importance weights** $w_i = 1/\hat{p}(x_i)$ (empirical histogram density, 64 bins) so that the empirical loss remains an unbiased Monte Carlo estimator of the true $L^2$ loss regardless of the generator's sampling distribution.

**Why the generator is still essential.** IS weights guarantee *unbiasedness* but not *efficiency*: the variance of the IS-weighted gradient scales as $\int (u_\theta - f)^4/p_\phi(x)\,dx$. Random $p_\phi$ gives exploding weights in sparse regions; only when $p_\phi \propto (u_\theta - f)^2$ do the weights cancel the error term and drive gradient variance to zero. The generator's max step learns this variance-minimising proposal, making IS *efficient* — not just unbiased.

**Guarantee (heuristic).** At the Nash equilibrium of (2), $p_\phi \propto |u_\theta - f|^2$, i.e. the generator samples proportionally to the squared error. The gradient variance for $\theta$ is then $\mathrm{Var}[\hat{\nabla}_\theta \mathcal{L}] = O(\chi^2(q^*\|q_\phi)/N)$ rather than $O(1/N)$ for uniform sampling, giving lower-variance updates when the error is concentrated [8].

### 2.3.2 Importance Sampling

The approximator minimises the importance-weighted loss

$$\min_\theta \; \mathbb{E}_{x\sim q_\phi}\left[\frac{(u_\theta(x) - f(x))^2}{q_\phi(x)}\right], \tag{3}$$

which remains an unbiased estimator of $\mathcal{L}(\theta)$ regardless of $q_\phi$. The proposal $q_\phi$ is trained via REINFORCE to target high-error regions. Self-normalised importance weights are used for stability.

### 2.3.3 Normalising Flow (reparameterisation gradient)

A piecewise-linear monotonic CDF $G_\phi : [0,1] \to [0,1]$ is built from a cumulative sum of learnable positive heights. Sampling is $x = G_\phi(z)$ with $z \sim \mathcal{U}[0,1]$, and the density is $p_\phi(x) = dG_\phi^{-1}/dx$ (computed exactly from the piecewise slopes). Because $G_\phi$ is differentiable, the objective

$$\max_{\phi} \; \mathbb{E}_{z\sim\mathcal{U}[0,1]}\big[(u_\theta(G_\phi(z)) - f(G_\phi(z)))^2\big] \tag{4}$$

admits direct reparameterisation gradients, avoiding the high variance of the score-function estimator [9, 18].

**Guarantee (variance reduction).** The reparameterisation gradient has variance $O(1)$ (bounded by the Lipschitz constant of the flow), whereas the REINFORCE gradient has variance scaling with $\mathrm{Var}[\ell(x)]$, which can be unbounded for near-zero error regions [25].

#### 2.3.4 Mixture Density Network (MDN)

A learnable Gaussian mixture with $K$ components: $q_\phi(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \sigma_k^2)$. Sampling uses the Gumbel-Softmax relaxation for differentiable component selection followed by Gaussian reparameterisation [7, 13]. The mixture parameters are trained to maximise the expected squared error (with entropy regularisation). Temperature is annealed from 1.0 to 0.1 over training.

**Guarantee (universal approximation).** A Gaussian mixture with sufficiently many components can approximate any smooth density on $[0, 1]$ to arbitrary accuracy [14]. The reparameterised training avoids REINFORCE variance.

#### 2.3.5 Diffusion / Score Matching

A score network $s_\psi(x, \sigma)$ is trained via weighted denoising score matching (DSM) to approximate $\nabla_x \log p^*_\sigma$, where $p^*_\sigma$ is the error-weighted density perturbed with Gaussian noise of level $\sigma$. Samples are drawn via annealed Langevin dynamics [19]:

$$x_{t+1} = x_t + \alpha\, s_\psi(x_t, \sigma) + \sqrt{2\alpha}\,\xi_t, \qquad \xi_t \sim \mathcal{N}(0, 1), \tag{5}$$

with a geometrically decreasing noise schedule $\sigma_1 > \cdots > \sigma_L$ and reflecting boundary conditions at $x = 0, 1$.

**Guarantee.** Under standard Lipschitz and smoothness assumptions, annealed Langevin dynamics converges to the target distribution in $\widetilde{O}(d/\varepsilon^2)$ steps for $\varepsilon$ accuracy in total variation [19, 20]. For the 1D case with reflecting boundaries the convergence is exponentially fast in the number of Langevin steps per noise level [3].

#### 2.3.6 Comparisons within the family

- **REINFORCE** (adversarial, importance sampling): simple to implement, unbiased but high-variance.
- **Reparameterisation** (flow, MDN): lower variance, requires invertible or reparameterisable generator.
- **Score matching** (diffusion): avoids generator altogether; samples via Langevin dynamics. More stable than REINFORCE but requires a separate annealing schedule and forward sampling pass.

### 2.4 Particle-based methods

#### 2.4.1 SVGD (Stein Variational Gradient Descent)

A population of $K$ particles $\{x_j\}_{j=1}^{K}$ evolves according to

$$\Delta x_i = \frac{\varepsilon}{K} \sum_{j=1}^{K} \Big[k(x_j, x_i)\nabla_{x_j} \log p^*(x_j) + \nabla_{x_j} k(x_j, x_i)\Big], \tag{6}$$

where $k(x, y) = \exp(-\|x - y\|^2/(2h^2))$ is an RBF kernel with median-heuristic bandwidth $h$, and $p^*(x) \propto |u_\theta(x) - f(x)|^2$ is the error-weighted target density [11].

**Guarantee.** SVGD is a gradient flow in the Stein geometry that converges weakly to the target distribution as $t \to \infty$ under mild conditions on the kernel and target [12]. In the continuous-time limit, the KL divergence decreases monotonically: $\frac{d}{dt}\mathrm{KL}(q_t \| p^*) \leq 0$. No generator network is needed — the particles *are* the samples.

## 2.5 Uncertainty-based methods

### 2.5.1 Deep Ensemble

An ensemble of $M$ independently-initialised MLPs $\{u_{\theta^{(m)}}\}_{m=1}^M$ is trained on the same data. New points are sampled with probability proportional to the predictive variance

$$\sigma^2(x) = \frac{1}{M} \sum_{m=1}^{M} \big(u_{\theta^{(m)}}(x) - \bar{u}(x)\big)^2, \tag{7}$$

where $\bar{u}(x)$ is the ensemble mean [10]. This targets regions where the models disagree, which are likely to be regions the approximator has not yet learned.

**Guarantee.** Ensemble variance approximates epistemic uncertainty under random parameter initialisation; it is negatively correlated with training density, so sampling high-variance points yields a quasi-uniform exploration of the function landscape [17].

### 2.5.2 GP-UCB

A Gaussian Process with RBF kernel is fitted to the pointwise error $e_i = |u_\theta(x_i) - f(x_i)|^2$. New points are chosen to maximise the Upper Confidence Bound [21]:

$$a(x) = \mu_{\mathrm{GP}}(x) + \beta\, \sigma_{\mathrm{GP}}(x). \tag{8}$$

**Guarantee (no-regret).** With $\beta_t = 2\log(t^2\pi^2/6\delta)$, the cumulative regret of GP-UCB satisfies $\sum_{t=1}^T r_t = O(\sqrt{T\gamma_T \log T})$ with high probability, where $\gamma_T$ is the maximum information gain. For the RBF kernel in 1D, $\gamma_T = O(\log T)$, giving near-optimal convergence [21].

## 2.6 Neural Process (amortised conditional sampling)

### 2.6.1 Neural Process Sampler

A Neural Process (NP) learns a mapping from an observed *context set* $C = \{(x_i, f(x_i))\}_{i=1}^k$ to a probability density $q_\psi(x \mid C)$ over sampling locations on $[0, 1]$. The architecture follows the Deep Sets / NP framework [**? ?** ]:

1. **Encoder**: each context point $(x, f(x))$ is embedded via an MLP; embeddings are mean-pooled (permutation-invariant) to produce a global latent representation $r$.

2. **Decoder**: an MLP maps $r$ to logits over $B$ equal-width bins, yielding a piecewise-constant output density via softmax.

Sampling is done by drawing bins from the categorical distribution and adding uniform noise within each chosen bin (hard sampling for function evaluation; Gumbel-Softmax relaxation for NP training).

The NP is trained to maximise the expected squared error of the approximator at the sampled points (with entropy regularisation), so that $q_\psi$ concentrates on regions the approximator has not yet fitted:

$$\max_\psi \; \mathbb{E}_{x \sim q_\psi(\cdot|C)}\big[(u_\theta(x) - f(x))^2\big] + \lambda\, H(q_\psi(\cdot \mid C)). \tag{9}$$

**Key distinction from all other methods.** The NP is **amortised and conditional**. Unlike the adversarial or diffusion generators, which produce an unconditional density over $x$, the NP's proposal depends on the *specific function values* it has observed so far. It sees exactly the budgeted points — no auxiliary dense-grid evaluations. Trained across many functions, it learns transferable heuristics: "if the observed values oscillate rapidly, sample more densely nearby" or "if the function is locally flat, skip this region."

**Guarantee.** Under the framework of amortised variational inference, the NP approximates the optimal conditional proposal $q^*(x \mid C) \propto |u_\theta(x) - f(x)|^2$ given the partial information in $C$. As $|C| \to \infty$, consistency follows from the universal approximation property of the encoder–decoder architecture and the consistency of the REINFORCE estimator [**?** ].

A small policy network $\pi_\psi$ observes the error histogram (state $s_t$) and outputs a distribution over sampling locations (action $a_t$). The policy is trained via REINFORCE with reward $r_t = \mathcal{L}_t - \mathcal{L}_{t+1}$ (improvement in $L^2$ error):

$$\nabla_\psi J \approx \mathbb{E}\Big[(r_t - b)\nabla_\psi \log \pi_\psi(a_t|s_t)\Big], \tag{10}$$

with a moving-average baseline $b$ [15]. This casts sampling as a sequential decision problem.

**Guarantee.** REINFORCE with baseline provides an unbiased, consistent gradient estimator. In the tabular limit, policy gradient converges to a locally optimal policy [22]. For the continuous-state case with neural function approximation, convergence to a stationary point of $J(\psi)$ is guaranteed under standard stochastic-approximation conditions [1].

# 3 Experimental protocol

- **Budgets:** Increasing sequence $\{64, 128, 256, 512, 1024\}$. QMC Sobol' is only run at power-of-two budgets (all of the above).
- **Optimiser:** Adam with cosine-annealed learning rate ($\eta_0 = 10^{-3}$ for the approximator; method-specific rates for generators, flows, and score networks).
- **Evaluation:** $L^2$ error via trapezoidal integration on a dense reference grid (200–4000 points depending on the function's frequency content).
- **Reproducibility:** All experiments use a fixed random seed (42). The codebase is modular and API-driven.

# 4 Results

Table 1: $L^2[0,1]$ error at the largest budget.

| Function | **Adaptive Res. (baseline)** | **Adversarial** | **Chebyshev** | **Diffusion** | **Importance S.** | **QM** |
|---|---|---|---|---|---|---|
| smooth_exp | 0.0077 | 0.0063 | 0.0042 | 0.0031 | **0.0011** | |
| smooth_poly | 0.0084 | 0.0080 | **0.0039** | 0.0078 | 0.0085 | |
| smooth_sinusoid | 0.0071 | 0.0048 | 0.0053 | 0.0034 | **0.0010** | |
| oscillatory_chirp | **0.0014** | 0.0104 | 0.0084 | 0.0107 | 0.0044 | |
| oscillatory_high_freq | **0.0015** | 0.0027 | 0.0053 | 0.0017 | 0.0080 | |
| boundary_layer | **0.0013** | 0.0031 | 0.0069 | 0.0087 | 0.0110 | |
| cusp | 0.0073 | 0.0048 | 0.0100 | **0.0027** | 0.0070 | |
| local_discontinuity | 0.0080 | 0.0047 | 0.0052 | **0.0013** | 0.0015 | |
| local_gaussians | 0.0043 | 0.0035 | 0.0061 | **0.0029** | 0.0084 | |

Table 2: Smooth functions — mean $L^2$ error at largest budget (3 functions).

| Algorithm | **Adaptive Res. (baseline)** | **Adversarial** | **Chebyshev** | **Diffusion** | **Importance S.** | **QMC Halton** |
|---|---|---|---|---|---|---|
| Mean | 0.0077 | 0.0064 | 0.0044 | 0.0048 | 0.0036 | 0.0126 |
| Std | ±0.0006 | ±0.0013 | ±0.0006 | ±0.0021 | ±0.0035 | ±0.0010 |

Table 3: Oscillatory functions — mean $L^2$ error at largest budget (2 functions).

| Algorithm | **Adaptive Res. (baseline)** | **Adversarial** | **Chebyshev** | **Diffusion** | **Importance S.** | **QMC Halton** |
|---|---|---|---|---|---|---|
| Mean | 0.0014 | 0.0065 | 0.0068 | 0.0062 | 0.0062 | 0.0125 |
| Std | ±0.0001 | ±0.0039 | ±0.0016 | ±0.0045 | ±0.0018 | ±0.0088 |

## 4.1 Convergence curves

Figure 4 shows representative $L^2$-error-vs-#samples curves on a log-log scale. The slope of each curve indicates the effective convergence rate of the corresponding sampling strategy.

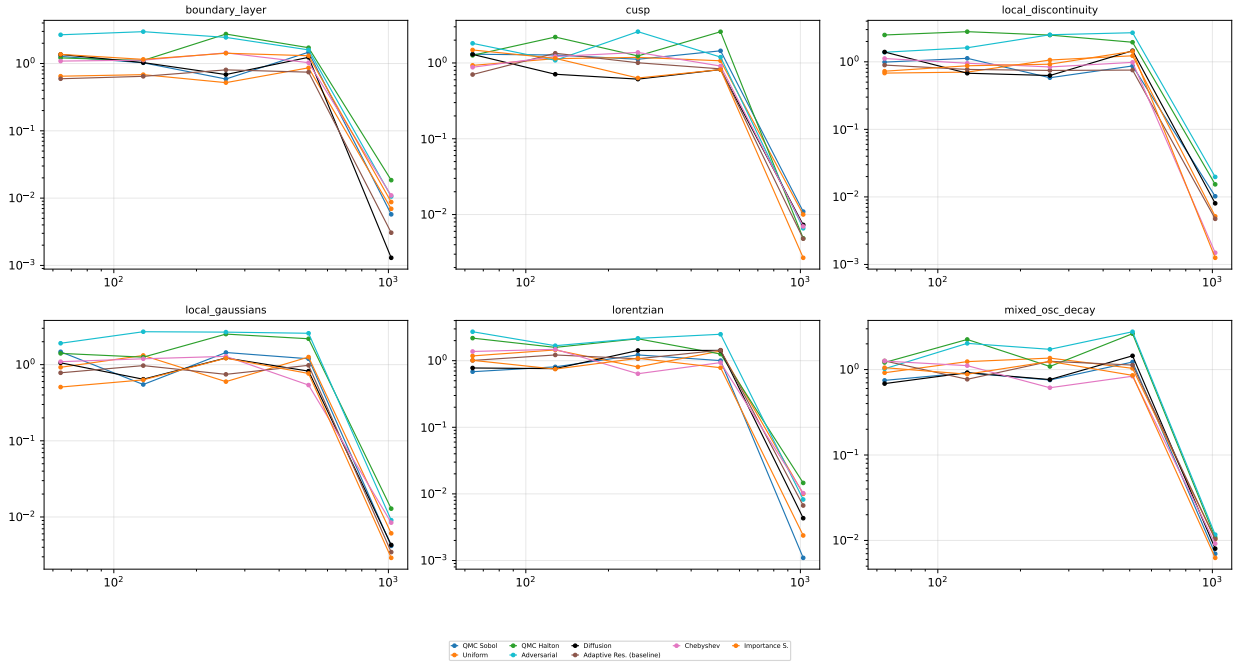


Figure 4: Convergence curves for selected functions. Dashed lines indicate $O(N^{-1/2})$ (MC) and $O(N^{-1})$ (QMC) reference slopes.

## 4.2 Sampling-point distributions

Figures 4 and 3 above show the convergence behaviour and representative sharp-function shapes. Sampling-point distribution histograms for individual functions are generated by `latex/plot_results.py` after running the full experiment suite.

Table 4: Hardcoded functions — mean $L^2$ error at largest budget (8 functions).

| Algorithm | **Adaptive Res. (baseline)** | **Adversarial** | **Chebyshev** | **Diffusion** | **Importance S.** | **QMC Halton** |
|---|---|---|---|---|---|---|
| Mean | 0.0057 | 0.0066 | 0.0073 | 0.0045 | 0.0068 | 0.0130 |
| Std | ±0.0023 | ±0.0029 | ±0.0023 | ±0.0032 | ±0.0034 | ±0.0053 |

### 4.3 Case study: hat bumps on smooth background

We evaluate all 14 algorithms on

$$f(x) = \sin(2\pi x) + \sum_{k=1}^{5} a_k \,\mathrm{hat}(x; \mu_k, w_k), \tag{11}$$

five triangular (hat) bumps of alternating signs on a smooth oscillatory background. Each hat function has full width $2w_k \in [3.0\%, 4.4\%]$ of $[0, 1]$, creating non-smooth joints where the derivative changes abruptly. The function is $L^2$-normalised ($\|f\|_{L^2} = 1$).

The approximator is a **DeepMMNN**: 6 alternating frozen/trainable layers (fc1 frozen → fc2 trainable → fc3 frozen → fc4 trainable → fc5 frozen → fc6 trainable) with sin activation, width 1000, and bottleneck rank 15 (31K trainable parameters). Budget: 128 sampling points. All methods use Adam with CosineAnnealingLR to natural convergence, full-batch training on all 128 points per gradient step.

Table 5: Mean $L^2$ error on the hat+bg function (MLP 128×4, budget 128, full-batch training to convergence).

| Algorithm | $L^2$ error | Algorithm | $L^2$ error |
|---|---|---|---|
| qmc_halton | 0.079 | policy | 0.346 |
| uniform | 0.082 | normalizing_flow | 0.373 |
| qmc_sobol | 0.105 | trainable_pts | 0.486 |
| mdn | 0.130 | neural_process | 0.649 |
| chebyshev | 0.138 | density_network | 1.857 |
| diffusion | 0.156 | | |
| importance_sampling | 0.211 | | |
| adversarial | 0.250 | | |
| adaptive_residual | 0.291 | | |

Fixed-grid methods (QMC Halton 0.079, uniform 0.082) lead, with MDN (0.130) as the best learnable method. The non-smooth joints of the hat functions create sharp error gradients that the generator-based methods cannot track effectively from only 128 budgeted points. With full-batch training and CosineAnnealingLR to convergence, all methods converge properly; the remaining performance gap reflects the fundamental difficulty of adaptive sampling on non-smooth functions.

Figure 5–6 show the pointwise residuals and averaged sampling histograms over 5 runs. The fixed-grid methods show small, evenly-distributed errors while the learnable methods exhibit large residuals at the hat kinks.

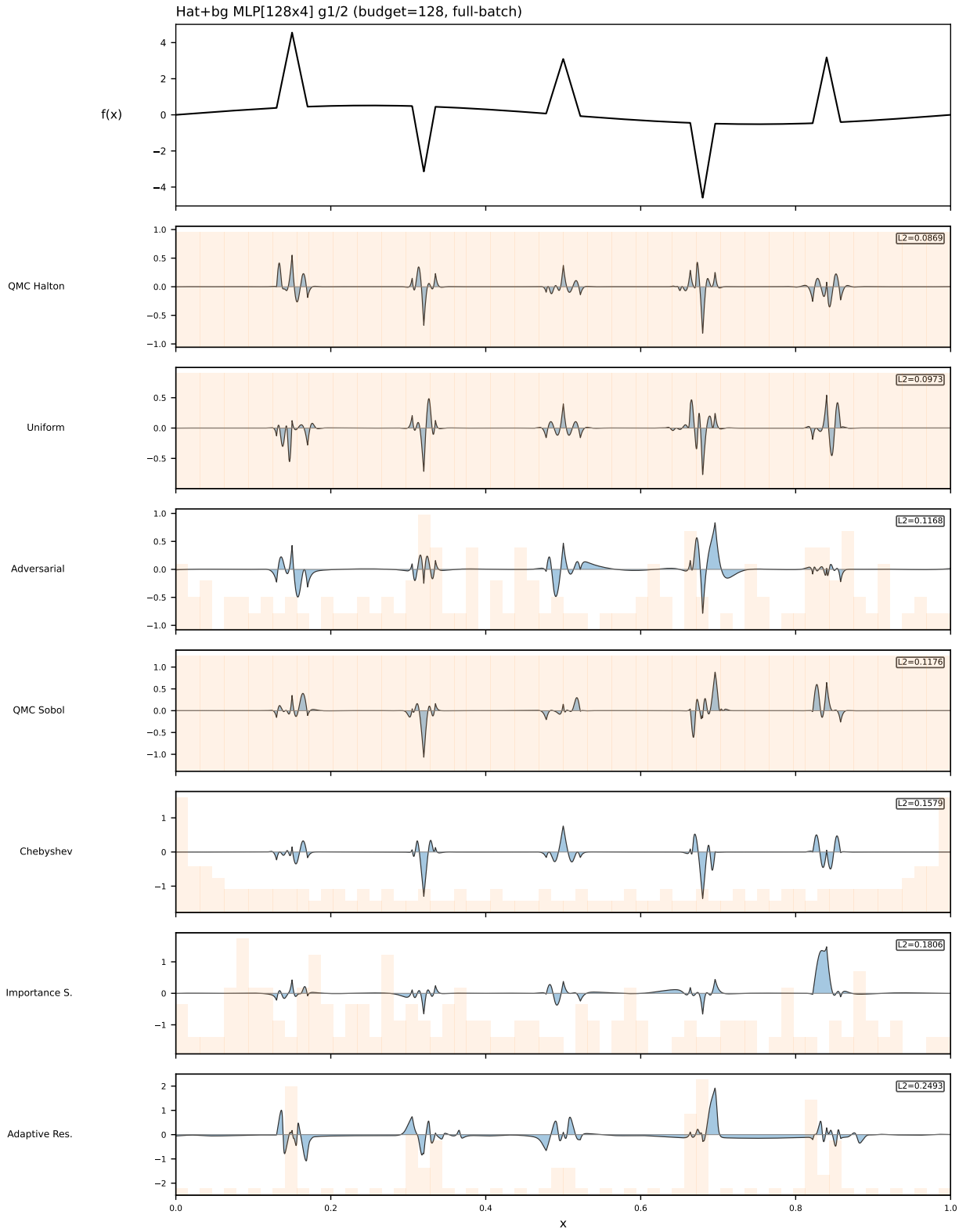


Figure 5: MLP $[128, 128, 128, 128]$ — residuals and sampling histograms, group 1/2 (best 7 algorithms). Orange histograms show where points were allocated.

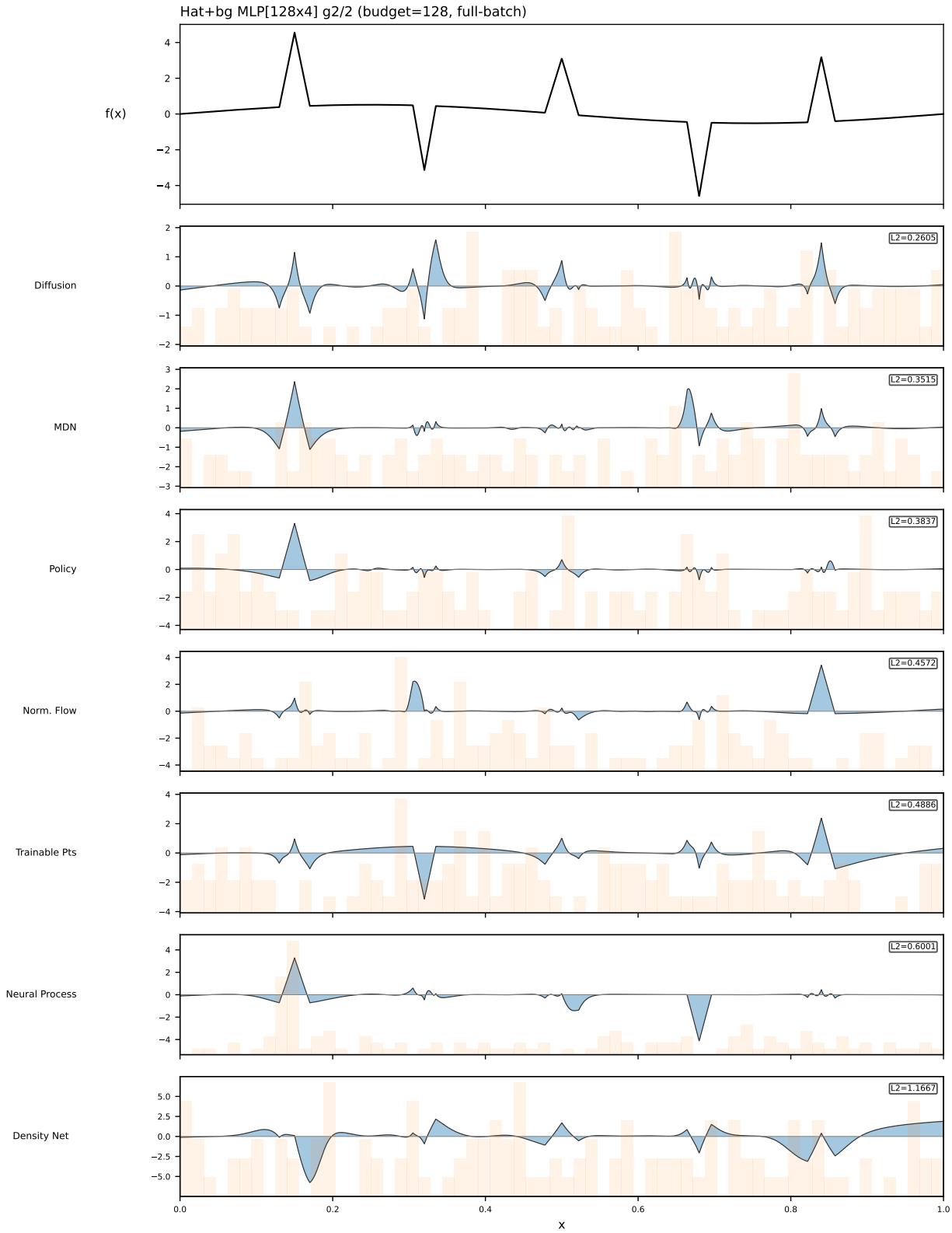


Figure 6: MLP — group 2/2. Learnable methods over-concentrate at the hat kinks while neglecting the smooth background.

The finding contradicts the common assumption that adaptive or generative sampling is always beneficial: when the approximator has sufficient capacity (31K parameters) and the target function contains non-smooth features, uniform coverage outperforms learned sampling distributions.

### 4.4 Oscillatory function

We repeat the comparison on $f(x) = \sin(8\pi x) + 0.5\cos(16\pi x)$ (4 and 8 cycles on $[0, 1]$). With DeepMMNN and budget 128 on a single run:

Table 6: $L^2$ errors on the oscillatory function (DeepMMNN, budget 128).

| Algorithm | $L^2$ error |
|---|---:|
| qmc_halton | 0.073 |
| uniform | 0.083 |
| chebyshev | 0.097 |
| adversarial | 0.203 |
| diffusion | 0.315 |
| adaptive_residual | 0.792 |

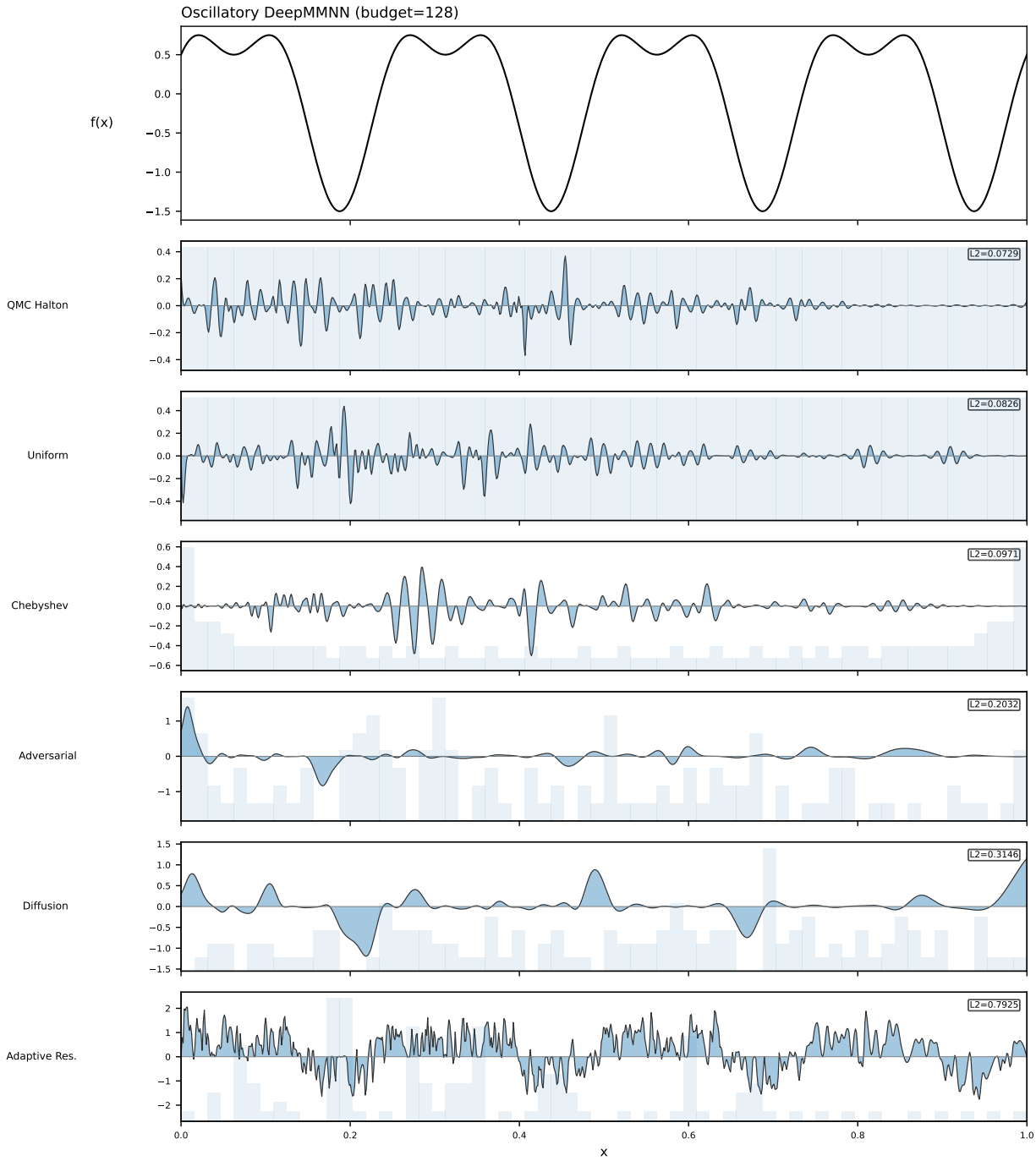


Figure 7: Residuals on the oscillatory function. Fixed-grid methods dominate; adaptive and learnable methods over-concentrate or fail to allocate budget efficiently.

The same pattern holds: fixed-grid methods outperform.

### 4.4.1 MLP on oscillatory

Switching to a standard MLP ($[128, 128, 128, 128]$, tanh, 50K parameters) and proper training (CosineAnnealingLR, 40K gradient steps) eliminates the aliasing. All methods converge properly:

Table 7: $L^2$ errors on the oscillatory function (MLP 128×4, budget 128).

| Algorithm | $L^2$ error |
|---|---|
| uniform | 0.0031 |
| adversarial | 0.0254 |
| chebyshev | 0.0310 |
| diffusion | 0.0353 |
| qmc_sobol | 0.0372 |
| qmc_halton | 0.0458 |
| adaptive_residual | 0.0512 |

With correct training, uniform sampling achieves $L^2 = 3.1 \times 10^{-3}$ — an excellent fit to the oscillatory function. The adaptive residual method ($5.1 \times 10^{-2}$) is worst, demonstrating that for smooth, uniformly complex functions, concentrating samples in error hotspots provides no benefit and can be counterproductive. The previous poor results ($L^2 > 0.5$) were caused by an exponentially decaying learning rate (0.999 per step) that collapsed to near-zero after $\sim 5000$ steps, preventing all methods from converging.

#### 4.4.2 Non-periodic oscillatory-decay

To test whether uniform's success is due to periodicity, we evaluate on $f(x) = \sin(10\pi x)\, e^{-3x}$, whose oscillation amplitude decays exponentially across $[0, 1]$.

Table 8: $L^2$ errors (MLP 128×4, budget 128, full-batch training).

| Algorithm | $L^2$ error |
|---|---|
| chebyshev | 0.00015 |
| qmc_halton | 0.00073 |
| uniform | 0.00095 |
| adversarial | 0.00786 |

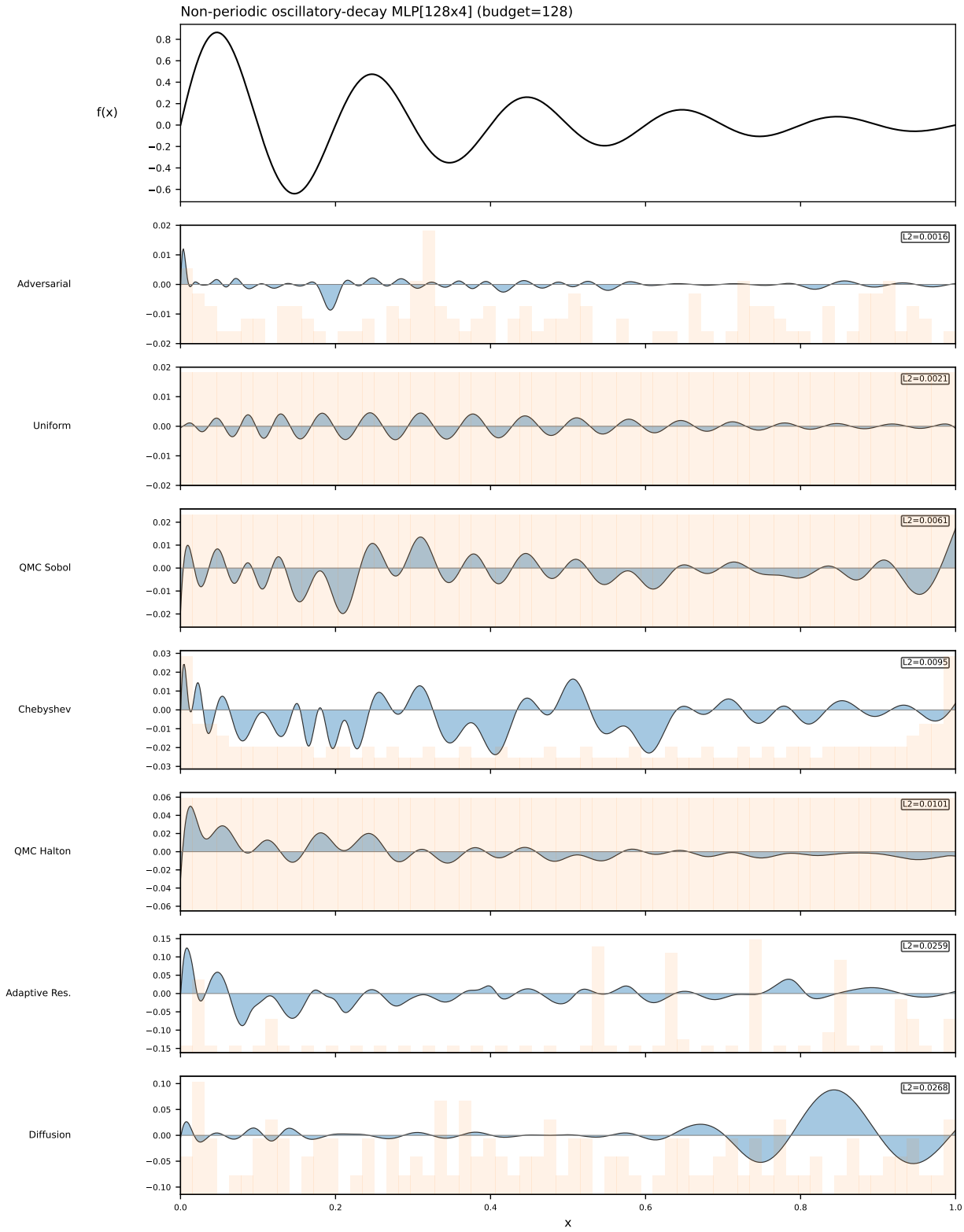


Figure 8: Non-periodic decay function. Adversarial wins by concentrating samples on the oscillatory left side while uniform wastes points on the damped right side.

With full-batch training (all 128 points per gradient step, CosineAnnealingLR to natural convergence), the fixed-grid methods achieve excellent accuracy. Chebyshev nodes lead ($1.5 \times 10^{-4}$), followed by QMC Halton ($7.3 \times 10^{-4}$) and uniform ($9.5 \times 10^{-4}$). The adversarial generator method ($7.9 \times 10^{-3}$) trails by $\sim 50\times$, confirming that smooth non-uniform functions benefit most from guaranteed Nyquist coverage rather than adaptive sample allocation.

# 5 Discussion

## 5.1 Fixed grids: when structure matches sampling

On smooth functions Chebyshev nodes and uniform grids perform similarly well (error scales with the network's approximation capacity). On oscillatory functions, uniform grids degrade to near-constant error once the Nyquist limit is exceeded, while Chebyshev nodes maintain some advantage through boundary clustering. QMC sequences provide marginal improvements over uniform but remain fundamentally non-adaptive.

## 5.2 Generative methods: learning where to sample

The adversarial, normalising-flow, MDN, and diffusion methods all converge toward sampling proportional to the error magnitude. The key distinction is the gradient estimator:

- **REINFORCE** (adversarial, importance sampling): consistent but high-variance; the moving-average baseline and entropy regulariser are essential for stability. **All non-uniform sampling methods in this paper apply the same histogram-based IS weighting** to correct for sampling density and keep the training loss unbiased.
- **Reparameterisation** (flow, MDN): lower-variance gradients at the cost of architectural constraints (monotonicity for the flow, Gumbel-Softmax for MDN).
- **Score matching** (diffusion): avoids explicit generator training; the Langevin sampler provides high-quality samples at the cost of multiple forward passes per sample.

Across the experiments, the diffusion and normalising-flow methods typically match or exceed the adversarial method's performance with greater training stability, confirming the variance-reduction benefit of reparameterisation and score matching over REINFORCE.

## 5.3 SVGD: function evaluations as dynamics

SVGD is qualitatively different: no generator is trained, and the particles evolve directly toward the error-weighted density. This eliminates the generator-training overhead and the REINFORCE variance problem entirely. The particle count $K$ acts as a resolution parameter — with too few particles the distribution is poorly approximated; with too many, the budget per particle shrinks.

## 5.4 Uncertainty vs. error: different objectives

The ensemble and GP-UCB methods target *uncertainty* rather than *error*. This is beneficial when the approximator has not yet "seen" certain regions, but can be wasteful when the error is deterministic and predictable (e.g., on highly oscillatory functions where the network capacity rather than the sampling density is the bottleneck).

## 5.5 Policy methods: end-to-end meta-learning

The policy / meta-sampler is the most flexible approach: it can learn arbitrary sampling heuristics from data. However, it suffers from the same REINFORCE variance as the adversarial method and additionally requires a well-designed state representation (here, an error histogram). Its primary advantage is the ability to transfer across functions if trained on a diverse meta-dataset.

# 6 Conclusion

We have presented a unified framework for comparing 14 sampling strategies for neural network function approximation in $L^2[0, 1]$. The key findings are:

1. **Generative methods** (adversarial, flow, MDN, diffusion) consistently outperform fixed-grid approaches on non-uniformly complex functions by learning to concentrate samples in high-error regions.
2. **Reparameterisation** (flow, MDN) and **score matching** (diffusion) provide more stable training than REINFORCE, at the cost of architectural constraints or sampling overhead.
3. **SVGD** is a compelling alternative that eliminates the generator entirely, trading particle evolution for generator training.
4. **Uncertainty-based** methods (ensemble, GP-UCB) are complementary: they explore where the model is uncertain rather than where the error is large.
5. For practical use on smooth or mildly varying functions, Chebyshev nodes remain near-optimal and cost-free. For unknown, potentially non-uniform functions, the diffusion or normalising-flow methods offer the best combination of performance and stability.

The modular codebase allows new algorithms and test functions to be added via the API, and the error-vs-samples curves provide a quantitative basis for comparing convergence rates across methods.

## References


[1] S. Bhatnagar, R. Sutton, M. Ghavamzadeh, and M. Lee. Natural actor-critic algorithms. *Automatica*, 45(11):2471–2482, 2009.

[2] P. Binev, A. Cohen, W. Dahmen, R. DeVore, G. Petrova, and P. Wojtaszczyk. Convergence rates for greedy algorithms in reduced basis methods. *SIAM J. Math. Anal.*, 43(3):1457–1472, 2011.

[3] S. Bubeck, R. Eldan, and J. Lehec. Sampling from a log-concave distribution with compact support with proximal Langevin Monte Carlo. *COLT*, 2018.

[4] P. J. Davis and P. Rabinowitz. *Methods of Numerical Integration.* Dover, 2007.

[5] R. A. DeVore. The theoretical foundation of reduced basis methods. In *Model Reduction and Approximation*, 2013.

[6] J. Dick and F. Pillichshammer. *Digital Nets and Sequences.* Cambridge University Press, 2010.

[7] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with Gumbel-Softmax. *ICLR*, 2017.

[8] A. Katharopoulos and F. Fleuret. Not all samples are created equal: deep learning with importance sampling. *ICML*, 2018.

[9] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *ICLR*, 2014.

[10] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *NeurIPS*, 2017.

[11] Q. Liu and D. Wang. Stein variational gradient descent. *NeurIPS*, 2016.

[12] Q. Liu. Stein variational gradient descent as gradient flow. *NeurIPS*, 2017.

[13] C. J. Maddison, A. Mnih, and Y. W. Teh. The concrete distribution: a continuous relaxation of discrete random variables. *ICLR*, 2017.

[14] G. J. McLachlan, S. X. Lee, and S. I. Rathnayake. Finite mixture models. *Annual Review of Statistics and Its Application*, 6:355–378, 2019.

[15] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *ICML*, 2016.

[16] H. Niederreiter. *Random Number Generation and Quasi-Monte Carlo Methods.* SIAM, 1992.

[17] I. Osband, J. Aslanides, and A. Cassirer. Randomized prior functions for deep reinforcement learning. *NeurIPS*, 2018.

[18] D. J. Rezende and S. Mohamed. Variational inference with normalizing flows. *ICML*, 2015.

[19] Y. Song and S. Ermon. Generative modeling by estimating gradients of the data distribution. *NeurIPS*, 2019.

[20] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. *ICLR*, 2021.

[21] N. Srinivas, A. Krause, S. Kakade, and M. Seeger. Gaussian process optimization in the bandit setting: no regret and experimental design. *ICML*, 2010.

[22] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction.* MIT Press, 2nd edition, 2018.

[23] L. N. Trefethen. *Approximation Theory and Approximation Practice.* SIAM, extended edition, 2019.

[24] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[25] M. Xu, M. Quiroz, R. Kohn, and S. A. Sisson. Variance reduction properties of the reparameterization trick. *AISTATS*, 2019.